Sisteme Distribuite

Laboratorul 3

Ceasuri

Andrei Pătrașcu

Răzvan Anghel

Ceasuri și sincronizare

Ceasurile sunt mecanisme hardware (cristale de quarț) care gestionează timpul în calculatoare, sub forma unor countere. Ele sunt alimentate chiar și când calculatorul este închis de către o baterie, astfel încât timpul să nu rămână în urmă când repornim calculatorul. Din păcate, nu toate cristalele sunt la fel, și ele vibrează la frecvențe diferite, ceea ce garantează că mai multe calculatoare vor avea timpi diferiți la un moment dat [1].

În sisteme de importanță maximă (bancare, securitate, gestionarea fișierelor etc.), este necesar ca nodurile unei rețele distribuite să fie sincronizate din punct de vedere al timpului, adică să ajungă la un consens fie din punct de vedere al ordinii desfășurării unor evenimente (ceasuri logice), fie din punct de vedere al valorii ceasurilor la desfășurarea unor evenimente (ceasuri fizice). UTC este standardul de măsurare a timpului prin 400 de ceasuri atomice, ale căror rezultate sunt transmise la nivel global prin sateliți, antene radio, protocolale la nivel de rețea și comunicație prin fibră optică [1].

Ideea de bază a sincronizării ceasurilor fizice este păstrarea deviației dintre noduri într-un anumit interval, numit precizie:

$$\forall t, \forall p, q : |C_p(t) - C_q(t)| \leq \pi,$$

unde p, q sunt calculatoare, t este timpul UTC iar π este precizia. De asemenea, ceasurile unui sistem distribuit mai pot fi sincronizate și folosind o sursă externă credibilă (un server special sau un ceas atomic).

Network Time Protocol

Network Time Protocol (NTP) are la bază algoritmul lui Cristian și este folosit pentru a sincroniza ceasuri prin rețea folosind un timeserver. Problema principală este că timpul raportat de server nu va mai fi corect datorită timpului de livrare al pachetelor respective. Algoritmul încearcă să estimeze trip time-ul folosind media aritmetică. Offset-ul este reprezentat de:

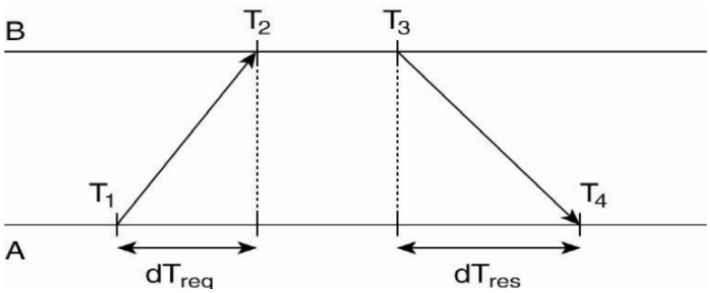


Figura 1: Algoritmul lui Cristian

$$\theta = T_3 + \frac{(T_2 - T_1) + (T_4 - T_3)}{2} - T_4 = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$$

Ceasuri logice Lamport

În multe aplicații este mai importantă ordinea evenimentelor, nu timpul la care acestea s-au desfășurat. Lamport a observat că două procese care nu comunică între ele nu au nevoie să își sincronizeze ceasurile, deoarece lipsa ei nu le afectează activitatea. Contează doar ordinea evenimentelor care se pot influența reciproc [1].

Astfel, Lamport definește relația *happens-before*, notată $a \to b$:

1. dacă a și b sunt evenimente din același proces și a are loc înaintea lui b, atunci $a \to b$;
2. dacă a este trimiterea unui mesaj, iar b este primirea aceluiași mesaj, atunci $a \to b$;
3. dacă $a \to b$ și $b \to c$, atunci $a \to c$ (tranzitivitate).

Dacă pentru două evenimente nu are loc nici $a \to b$, nici $b \to a$, spunem că ele sunt *concurente*, ceea ce nu înseamnă că s-au petrecut simultan, ci doar nu se pot influența unul pe celălalt. Ceasurile Lamport atribuie fiecărui eveniment a o valoare $C(a)$ astfel încât:

$$a \to b \Rightarrow C(a) < C(b).$$

Algoritmul lui Lamport

Fiecare proces P_i păstrează un contor local C_i, inițializat cu 0. Acesta nu are legătură cu timpul real, ci doar numără evenimente. Regulile sunt:

1. Înainte de fiecare eveniment, procesul își incrementează contorul: $C_i = C_i + 1$.
2. Când P_i trimite un mesaj m, trimite și un timestamp $ts(m) = C_i$ (valoarea de după incrementare).
3. Când P_j primește mesajul m, își actualizează contorul $C_j = \max(C_j, ts(m))$, apoi îl incrementează cu 1 pentru a simboliza primirea mesajului.

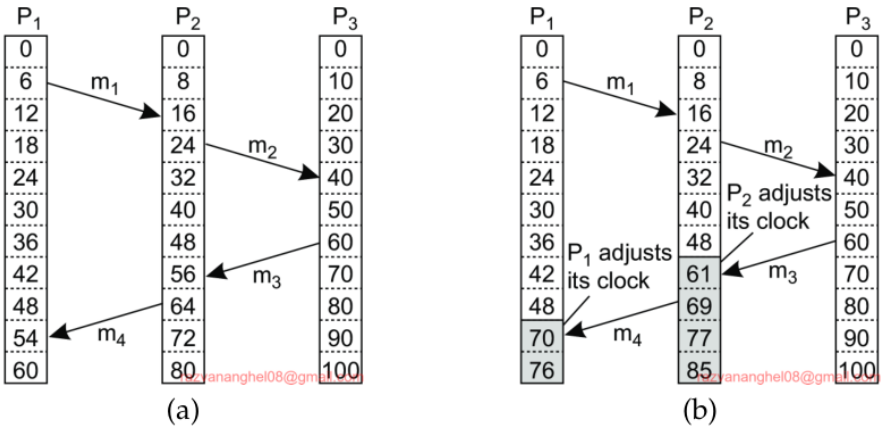


Figura 2: Ceasuri logice Lamport: (a) trei procese cu ceasuri care au frecvențe diferite; (b) ceasurile sunt corectate cu algoritmul lui Lamport

În figura (a), m_3 ajunge la destinație cu o valoare a ceasului *mai mică* decât cea la care a fost trimis ($60 \to 56$). Acest lucru este imposibil, deoarece un mesaj nu poate fi primit înainte de a fi trimis.

În figura (b), P_2 observă că $ts(m)$ este mai mare decât propriul contor și își incrementează ceasul la $ts(m) + 1$, ceea ce asigură ordinea corectă.

Algoritmul garantează că dacă $a \to b$, atunci $C(a) < C(b)$, dar **reciproca nu este adevărată**: din $C(a) < C(b)$ nu putem deduce că $a \to b$, deoarece două evenimente concurente pot avea oricare ordine a valorilor. Prin urmare, ceasurile Lamport ne permit să ordonăm evenimentele cauzale, dar nu să detectăm concurența [1].

Ceasuri vectoriale

Ceasurile vectoriale rezolvă această problemă, păstrând mai multe informații [1]. Într-un sistem cu N procese, fiecare proces P_i păstrează un vector VC_i de N contoare, inițializat cu zerouri:

- $VC_i[i]$ este numărul de evenimente petrecute în P_i;
- $VC_i[k]$, cu $k \neq i$, este numărul de evenimente din P_k care s-au petrecut înaintea evenimentului curent din P_i, adică evenimente care l-ar fi putut influența.

Algoritmul

Regulile sunt o extensie a celor de la ceasurile Lamport:

1. Înainte de fiecare eveniment (intern sau transmiterea unui mesaj), procesul P_i își incrementează propria componentă: $VC_i[i] = VC_i[i] + 1$.
2. Când P_i trimite un mesaj m, îi atașează întregul vector, deci $ts(m) = VC_i$ (după incrementare).
3. Când P_j primește mesajul m, actualizează fiecare componentă a vectorului cu maximul dintre valoarea proprie și cea din mesaj:

$$VC_j[k] = \max(VC_j[k], ts(m)), \quad \forall k \in [1, N],$$

apoi își incrementează propria componentă, $VC_j[j] = VC_j[j] + 1$.

Pentru două evenimente a și b cu vectorii $VC(a)$ și $VC(b)$ definim:

- $VC(a) \leq VC(b)$ dacă $VC(a)[k] \leq VC(b)[k], \forall k \in [1, N]$;
- $VC(a) < VC(b)$ dacă $VC(a) \leq VC(b)$ și $VC(a) \neq VC(b)$.

Astfel, cu ceasurile vectoriale putem deduce evenimente concurente pe baza valorilor contoarelor:

$$a \to b \iff VC(a) < VC(b).$$

Dacă nu are loc nici $VC(a) < VC(b)$, nici $VC(b) < VC(a)$, atunci evenimentele a și b sunt **concurente**. În acest caz, unele componente ale lui $VC(a)$ sunt mai mari, iar altele mai mici decât cele ale lui $VC(b)$.

Exemplu

Considerăm trei procese, P_1, P_2, P_3, și următoarele evenimente:

- a: P_1 trimite mesajul m_1 către P_2;
- b: P_2 primește mesajul m_1;
- c: eveniment intern în P_3;
- d: P_2 trimite mesajul m_2 către P_3;
- e: P_3 primește mesajul m_2.

Eveniment	Proces	Ceas Lamport	Ceas vectorial
a	P_1	1	$(1,0,0)$
b	P_2	2	$(1,1,0)$
c	P_3	1	$(0,0,1)$
d	P_2	3	$(1,2,0)$
e	P_3	4	$(1,2,2)$

Tabela 1: Ceasuri Lamport și vectoriale pentru exemplul cu trei procese

De exemplu, la evenimentul e, procesul P_3 avea $VC_3 = (0,0,1)$, iar mesajul m_2 aduce $ts(m_2) = (1,2,0)$. Maximul pe componente este $(1,2,1)$, iar după incrementarea componentei proprii obținem $(1,2,2)$.

Din tabel putem observa că:

- $(1,0,0) < (1,2,2)$, deci $a \to e$: mesajul trimis de P_1 a influențat, indirect, evenimentul e;
- evenimentele b și c au vectorii $(1,1,0)$ și $(0,0,1)$ și nu se pot compara (prima componentă este mai mare la b, a treia la c), deci sunt **concurente**.

Pentru ceasurile vectoriale costul detectării concurenței vine cu o dimensiune mai mare a mesajelor, N în loc de 1, comparativ cu ceasurile Lamport. Ele sunt folosite, de exemplu, la detectarea actualizărilor concurente în baze de date replicate [1].

Exerciții

1. Un client cu ceasul decalat cere de un server timpul corect printr-o rețea cu întârzieri simulate cu `time.Sleep`. Pe lângă θ, vom folosi și durata round-trip-ului, fără timpul de procesare al serverului: **2p**

$$\delta = (T_4 - T_1) - (T_3 - T_2).$$

Pornind de la șablonul de mai jos:

```
// stocheaza deviatia fata de timpul real
type Clock struct{ offset time.Duration }

func (c *Clock) Now() time.Time { return time.Now().Add(c.offset) }

// TODO:
// Calculeaza decalajul theta si round-trip-ul delta
func estimate(t1, t2, t3, t4 time.Time) (theta, delta time.Duration) {
    return 0, 0
}

func measure(c_clock, s_clock *Clock) (theta, delta time.Duration) {

    // Request
    t1 := c_clock.Now()
    time.Sleep(3*time.Millisecond + time.Duration(rand.Intn(10))*time.
Millisecond)

    t2 := s_clock.Now()
    // Procesare intre 5 si 20 ms
    time.Sleep(5*time.Millisecond + time.Duration(rand.Intn(20))*time.
Millisecond)
```

```
    t3 := s_clock.Now()

    // Response
    time.Sleep(3*time.Millisecond + time.Duration(rand.Intn(10))*time.
Millisecond)
    t4 := c_clock.Now()

    return estimate(t1, t2, t3, t4)
}
```

(a) Completați funcția `estimate` folosind Algoritmul lui Cristian. Simulați apoi în *main* o cerere și un răspuns folosind funcția measure și verificați dacă offset-ul inițial a scăzut.
(b) Repetați simularea de 10 ori și calculați δ și θ minime. Comparați rezultatele cu cele obținute la punctul *a)*.

2. Trei goroutine își trimit mesaje între ele prin canale. Ele își trimit mesaje conform topologiei inel $P_1 \to P_2 \to P_3 \to P_1$. Fiecare dintre aceste evenimente este înregistrat folosind ceasuri logice Lamport. Pornind de la șablonul de mai jos, implementați cele 3 metode ale ceasului și stocați în main evenimentele ordonate crescător. Puteți folosi *sort.Slice* cu funcția *less*. Între evenimente, contoarele ceasurilor sunt incrementate aleator. **3p**

```
type LamportClock struct{ counter int }

// incrementeaza contorul
func (l *LamportClock) Tick() {}
func (l *LamportClock) Send() {}
func (l *LamportClock) Receive(ts int) {}

type Message struct{ sender, ts int }
type Event struct {
    process          int
    internal_counter int
    kind             string // "internal", "send", "recv"
}

// TODO: retuneaza true daca a precede b
func less(a, b Event) bool { return false }
```

Bibliografie

[1] Maarten van Steen and Andrew S. Tanenbaum. *Distributed Systems*. Maarten van Steen, 4th edition, 2023.